

Newton

牛顿迭代法

引入

本文介绍如何用牛顿迭代法（Newton's method for finding roots）求方程的近似解，该方法于 17 世纪由牛顿提出。

具体的任务是，对于在 $\mathbb{R}$ 上连续且单调的函数 f ，求方程 $f(x) = 0$ 的近似解。

解释

初始时我们从给定的 x_0 和一个近似解 r 开始（初值的问题与 Newton 分形有关，可参考 3Blue1Brown 的[牛顿分形](#)）。

假设我们目前的近似解是 x_n ，我们画出与 f 切于点 $(x_n, f(x_n))$ 的直线，将 f 与 x 轴的交点横坐标记为 x_{n+1} ，那么这就是一个更优的近似解。重复这个迭代的过程。根据导数的几何意义，可以得到如下关系：

整理后得到如下递推式：

直观地说，如果 f 比较平滑，那么随着迭代次数的增加， x_n 会越来越逼近方程的解。

牛顿迭代法的收敛率是平方级别的，这意味着每次迭代后近似解的精确数位会翻倍。关于牛顿迭代法的收敛性证明可参考 [citizendium - Newton method Convergence analysis](#)

当然牛顿迭代法也同样存在着缺陷，详情参考 [Xiaolin Wu - Roots of Equations 第 18 - 20 页分析](#)

求解平方根

我们尝试用牛顿迭代法求解平方根。设 $f(x) = x^2 - n$ ，这个方程的近似解就是 $\sqrt{n}$ 的近似值。于是我们得到

在实现的时候注意设置合适的精度。代码如下

实现



C++Python

```
1 double sqrt_newton(double n) {
2     constexpr static double eps = 1E-15;
3     double x = 1;
4     while (true) {
5         double nx = (x + n / x) / 2;
6         if (abs(x - nx) < eps) break;
7         x = nx;
8     }
9     return x;
10 }
```

```

1 def sqrt_newton(n):
2     eps = 1e-15
3     x = 1
4     while True:
5         nx = (x + n / x) / 2
6         if abs(x - nx) < eps:
7             break
8         x = nx
9     return x

```

求解整数平方根

尽管我们可以调用 `sqrt()` 函数来获取平方根的值，但这里还是讲一下牛顿迭代法的变种算法，用于求不等式 的最大整数解。我们仍然考虑一个类似于牛顿迭代的过程，但需要在边界条件上稍作修改。如果 在迭代的过程中上一次迭代值得近似解变小，而这一次迭代使得近似解变大，那么我们就进行这次迭代，退出循环。

实现



C++Python

```

1 int isqrt_newton(int n) {
2     int x = 1;
3     bool decreased = false;
4     for (;;) {
5         int nx = (x + n / x) >> 1;
6         if (x == nx || (nx > x && decreased)) break;
7         decreased = nx < x;
8         x = nx;
9     }
10    return x;
11 }

```

```

1 def isqrt_newton(n):
2     x = 1
3     decreased = False
4     while True:
5         nx = (x + n // x) // 2
6         if x == nx or (nx > x and decreased):
7             break
8         decreased = nx < x
9         x = nx
10    return x

```

高精度平方根

最后考虑高精度的牛顿迭代法。迭代的方法是不变的，但这次我们需要关注初始时近似解的设置，即 的值。由于需要应用高精度的数一般都非常大，因此不同的初始值对于算法效率的影响也很大。一个自然的想法就是考虑 ，这样既可以快速计算出 ，又可以较为接近平方根的近似解。

实现

给出 Java 代码的实现：

```
1 public static BigInteger isqrtNewton(BigInteger n) {
2     BigInteger a = BigInteger.ONE.shiftLeft(n.bitLength() / 2);
3     boolean p_dec = false;
4     for (;;) {
5         BigInteger b = n.divide(a).add(a).shiftRight(1);
6         if (a.compareTo(b) == 0 || a.compareTo(b) < 0 && p_dec)
7             break;
8         p_dec = a.compareTo(b) > 0;
9         a = b;
10    }
11    return a;
12 }
```

实践效果：在 的时候该算法的运行时间是 60 ms，如果我们不优化 的值，直接从 开始迭代，那么运行时间将增加到 120 ms。

习题

- [UVa 10428 - The Roots](#)
- [LeetCode 69. x 的平方根](#)

本页面主要译自博文 [Метод Ньютона \(касательных\) для поиска корней](#) 与其英文翻译版 [Newton's method for finding roots](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

本页面最近更新：2024/10/9 22:38:42，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [Marcythm](#), [allenanswerzq](#), [countercurrent-time](#), [Enter-tainer](#), [H-J-Granger](#), [hly1204](#), [lr1d](#), [Menci](#), [NachtgeistW](#), [nutshellfool](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#), [shawlleyw](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用